

Reading Project Report: ADDS (Asynchronous Dynamic Delta-Stepping)

Subject: Parallel Computing

Topic: GPU-Accelerated Single-Source Shortest Path (SSSP) Algorithms

Paper Reference: ADDS: Asynchronous Dynamic Delta-Stepping for GPU SSSP

1. Introduction

The Single-Source Shortest Path (SSSP) problem is a fundamental cornerstone of graph theory and parallel computing. Given a graph $G = (V, E)$ with non-negative edge weights and a starting source vertex s , the goal is to find the shortest path from s to every other vertex in the graph. SSSP is critical in applications ranging from GPS navigation and network routing to social network analysis and bioinformatics.

With the explosion of “Big Data”, graphs now contain billions of edges, necessitating the use of high-performance accelerators like GPUs. However, implementing SSSP on GPUs is notoriously difficult due to the irregular nature of graph data, which leads to load imbalances and divergent execution paths. Traditional parallel SSSP algorithms, such as the Near-Far approach or standard Delta-Stepping, often suffer from synchronization overheads and suboptimal parameter selection.

This report analyzes the ADDS (Asynchronous Dynamic Delta-Stepping) framework, which addresses these limitations by introducing an asynchronous execution model, multi-bucket priority management, and a dynamic mechanism for adjusting the “delta” (Δ) parameter to optimize performance across diverse graph topologies.

2. Problem Description and Motivation

2.1 The Limitations of Existing Approaches

The SSSP problem is typically solved using two classical algorithms: Dijkstra’s Algorithm (work-efficient but sequential) and the Bellman-Ford Algorithm (highly parallel but work-inefficient). Parallel SSSP research often focuses on Delta-Stepping, which strikes a balance between the two by dividing vertices into “buckets” based on their current tentative distance.

The provided notes highlight several flaws in existing GPU-based SSSP implementations, particularly the common “Near-Far” optimization:

- Coarse Granularity: Near-Far typically uses only two buckets (near and far). This lack of granularity often results in processing many vertices that are not yet ready for their final distance, leading to redundant edge relaxations.

- **Synchronization Barriers:** Most GPU implementations rely on “barrier synchronization” and “double buffering.” As noted on page 1 of the study notes, even if some threads become idle during a round, they cannot proceed to the next iteration’s nodes until all threads reach the barrier. This is particularly detrimental to “high-diameter” graphs (like road networks), which require thousands of iterations.
- **Static Delta Selection:** The performance of Delta-Stepping is highly sensitive to the value of Δ . A small Δ increases synchronization frequency, while a large Δ turns the algorithm into Bellman-Ford, increasing redundant work. Existing frameworks often require manual tuning of Δ , which does not adapt to the graph’s characteristics during runtime.

2.2 The “Delta” Trade-off

As illustrated in the graphs on page 4 of the notes (e.g., rmat22, road-CAL), there is a distinct “Clip Point” and a “Best Performance Point.”

If Delta is too small, items with significantly different distances are forced into the same final bucket (“clipping”), or the system spends too much time managing empty buckets.

If Delta is too large, parallelism increases, but work efficiency drops as the same vertex is relaxed multiple times.

3. The ADDS Architecture and Techniques

To solve these issues, the ADDS framework introduces a “Manager-Worker” thread block model on the GPU. This separates the high-level control logic from the low-level data processing.

3.1 Manager and Worker Thread Blocks (MTB & WTB)

ADDS utilizes a unique decomposition of labor:

- **MTB (Manager Thread Block):** A single thread block responsible for high-level management. It reads the worklist buckets, assigns work items to workers, manages memory, and—crucially—dynamically adjusts the Δ and the number of active buckets.
- **WTB (Worker Thread Blocks):** Multiple thread blocks that perform the heavy lifting. They relax edges, calculate new distances, and write updated vertices back into the appropriate buckets.

This separation allows for Asynchronous Execution. Unlike traditional models where every thread must participate in global synchronization, the MTB handles the “steering” while WTBs continue to “drive,” eliminating the need for global barriers.

3.2 Data Structures and Memory Management

ADDS manages work using a Circular Priority Queue consisting of multiple buckets (typically 32).

- Assignment Flag (AF): To avoid contention, each WTB polls its own AF in scratchpad memory. The MTB writes the location and size of the assigned work into this flag.
- Worklist (Buckets): Each bucket is a 32-bit array managed as a circular FIFO queue.
- Dynamic Memory Allocation: To handle the unpredictable growth of buckets, ADDS implements a block-based memory system (64K 32-bit words per block). It uses a “cache” in the GPU’s shared memory (scratchpad) to store address mappings, reducing the overhead of global memory lookups.

3.3 The Asynchronous Mechanism

The notes specify two key counters for managing data consistency without barriers:

- WCC (Write Completed Counter): Each segment has a WCC. A WTB must perform a memory fence and an atomic increment of the WCC after writing. The MTB only reads segments where the WCC indicates completion, ensuring it never processes “half-written” or out-of-order data.
- CWC (Completed Work Counter): This tracks when a block of memory is fully processed, allowing the MTB to recycle that memory block for future use.

4. Key Algorithms: Dynamic Delta-Stepping

The most significant contribution of ADDS is the Dynamic Adjustment of the Δ parameter during execution.

4.1 The Goal of Dynamic Delta

The goal is to maintain the system near the “best-perf-point” (the point where execution time is minimized). The MTB monitors the state of the GPU and the distribution of work:

- Lower Bound: The system tries to ensure the “tail bucket” contains at least 65% of the assigned work items (an empirical heuristic).
- GPU Utilization: The MTB monitors the number of active work items. If the GPU is under-utilized, it increases Δ to pull more vertices into the “near” buckets, increasing parallelism. If the GPU is saturated and work efficiency is dropping (too many redundant relaxations), it decreases Δ .

4.2 Avoiding Oscillation

Changing Δ too frequently can lead to instability. The framework implements a settling time (proportional to the current Δ) to allow the system to stabilize after an adjustment before making another change.

4.3 Load Balancing (Warp-Level Cooperation)

As noted on page 6, ADDS handles “Power Law” graphs (where a few vertices have a massive number of edges) through warp-level primitives:

- Sub-threshold nodes: Processed by a single warp.
- Super-threshold nodes: If a node’s degree exceeds a certain limit (`tb_coop_threshold`), multiple warps or even an entire thread block cooperate to relax its edges. This prevents a single “heavy” vertex from stalling the entire pipeline.

5. Related Works and Comparisons

5.1 Synchronous Delta-Stepping (Meyer & Sanders)

The original Delta-Stepping algorithm is inherently synchronous. It processes all vertices in bucket i before moving to bucket $i + 1$. In a parallel environment, this requires a global sync after every bucket. While effective for CPUs, the cost of global synchronization on GPUs is much higher.

5.2 Gunrock (Near-Far)

Gunrock is a popular GPU graph processing library. Its SSSP implementation uses a simplified Delta-Stepping known as Near-Far. While highly optimized, it still relies on a static Δ and a coarse two-bucket system. ADDS outperforms this by using a fine-grained, multi-bucket system that dynamically adapts to the “shape” of the shortest-path tree as it grows.

5.3 Asynchronous Bellman-Ford

Some approaches ignore buckets entirely and just let threads relax any updated vertex (asynchronous Bellman-Ford). While this maximizes GPU utilization, it results in terrible work efficiency, as many vertices are processed hundreds of times before their true shortest path is found. ADDS sits in the “sweet spot” between the strict order of Dijkstra and the chaotic parallelism of Bellman-Ford.

6. Evaluation and Performance Analysis

Based on the benchmark results provided in the notes (using Lonestar 4.0 and an RTX 2080 Ti):

6.1 Work Efficiency vs. Parallelism

The graphs for rmat22 (a synthetic graph) and road-CAL (a road network) show how ADDS navigates the performance landscape.

- In Road Networks, the diameter is high. A static, large Δ leads to “9x more work” (as seen in page 4, graph b). ADDS’s ability to keep the Δ small for these graphs saves significant computation time.
- In Power Law Graphs (like rmat), the diameter is low. Parallelism is the priority. ADDS identifies this and increases Δ to saturate the GPU’s streaming multiprocessors (SMs).

6.2 Results Summary

The study demonstrates that by removing the barrier synchronization and dynamically tuning Δ , ADDS can achieve significant speedups over both synchronous Delta-Stepping and the standard Near-Far approach. Specifically, for high-diameter graphs where the number of iterations is large, the asynchronous nature of ADDS allows the GPU to stay busy without waiting for the slowest thread in every round.

7. Conclusion

The ADDS framework represents a major step forward in parallel SSSP algorithms for GPUs. By moving away from the “one-size-fits-all” approach of static parameters and synchronous iterations, it acknowledges the reality of modern graph data: it is irregular, unpredictable, and diverse.

The introduction of the MTB/WTB Manager-Worker model provides a blueprint for other graph algorithms (like BFS or Betweenness Centrality) to implement asynchronous control on GPUs. Furthermore, the Dynamic Delta mechanism proves that “self-tuning” algorithms can significantly outperform manually tuned ones by adapting to the local characteristics of the graph during the search process.

As graph sizes continue to scale, the techniques pioneered in ADDS—asynchrony, multi-bucket prioritization, and dynamic load balancing—will be essential for maintaining the performance of parallel graph processing systems.

8. Future Work and Optimization Points

Based on the “Problem” and “Optimization” sections (page 5 and 7):

- Kernel Fusion: Exploring whether the Manager and Worker logic can be further fused to reduce memory traffic.
- Multi-GPU Support: Partitioning large graphs across multiple GPUs while maintaining the asynchronous bucket management across device boundaries.

- Atomic Contention: Improving the `resv_ptr` logic to reduce atomic collisions in high-degree vertex processing.
- **Adding PRO optimization from another paper. Have been done in my coding project.**